

BowlMania

Ansh Shah (1011273426), Bilaal Khan (1010859563)

1.0 Project Description

BowlMania is a Wii Sports-Bowling-inspired 10-frame bowling game rendered on the DE1-SoC that combines live VGA graphics, motion based controls, and audio playback into an interactive retro arcade experience. The player aims with buttons on a physical remote, performs a swing using an MPU6050 accelerometer, and the game maps that motion to ball power and lane drift. A finite state machine controls gameplay flow. Players control a Mii-style character who rolls a bowling ball down a perspective-rendered lane at a rack of 10 pins, with full scoring rules including strikes, spares, and 10th-frame bonus balls.

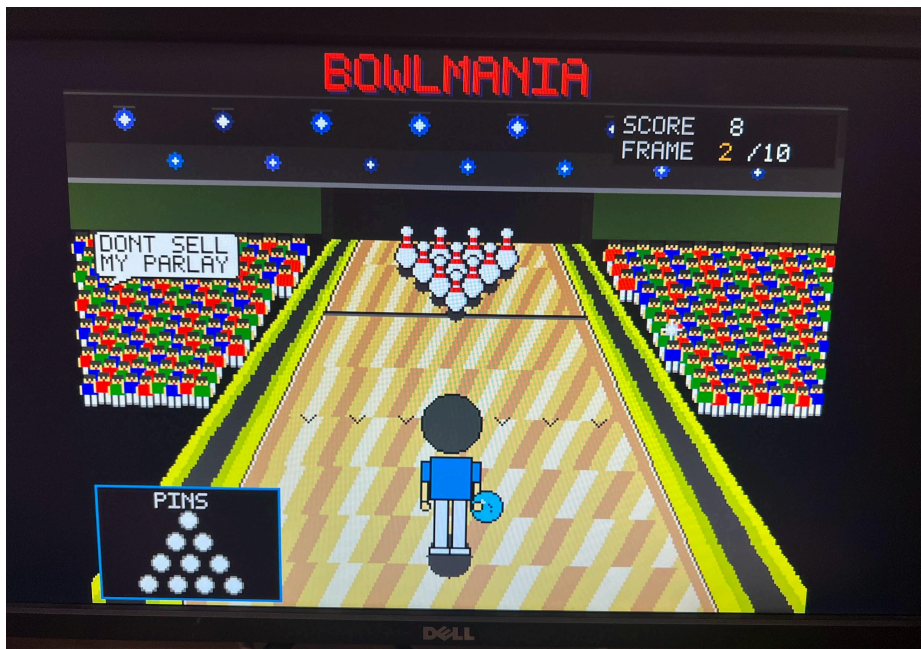


Figure 1: VGA display during gameplay showing scoreboard, lane, pins, character & crowd

The visual experience includes a perspective-correct animated lane with gutter geometry, animated disco ceiling lights, a Mii-style character, crowd fans with randomized speech bubbles and camera flash effects, a live power bar and arming indicator during the throw sequence, and a complete 10-frame scorecard HUD. The game supports a full start screen, automatic frame progression, and a game-over / restart flow.

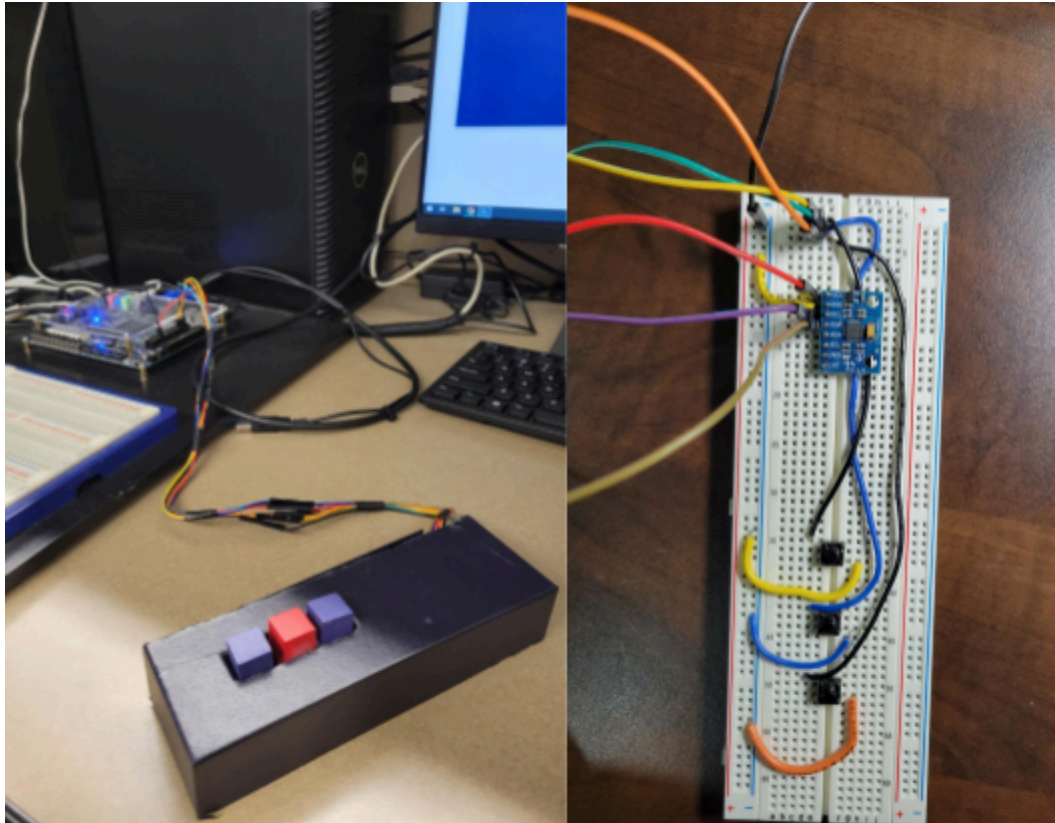
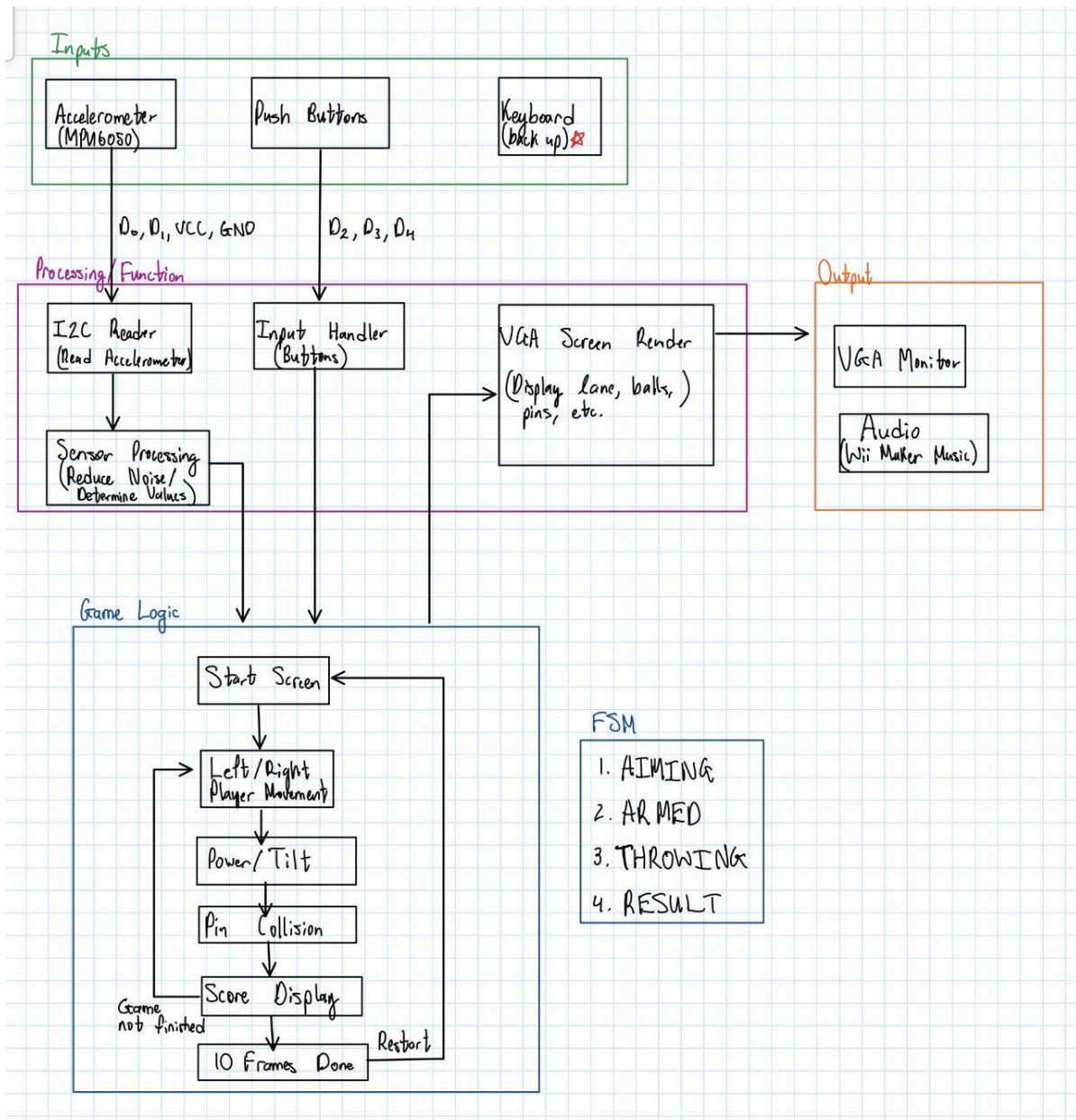


Figure 2: Physical Wii remote & it's circuit beneath

The primary input device is a GY-521 MPU-6050 accelerometer connected via bit-banged I2C over the JP1 GPIO expansion header. The player physically swings the remote to set throw power and lateral angle, mirroring the real-world motion of bowling. Left/right lane positioning is controlled by pushbuttons connected to the JP1. The game also drives the DE1-SoC audio peripheral to play sound samples on game events and background music.

2.0 Block Diagram



3.0 How to Operate

3.1 Game Controls

Input	Action
Hold Left Pushbutton (JP1)	Move player left

D3)	
Hold Right Pushbutton (JP1 D4)	Move player right
Hold/Release Middle Pushbutton (JP1 D2)	Holding keeps the game in armed state and reads the accelerometer for power and ball tilt angle.
Physical Swing of Remote	Swing power determines ball speed and tilt of the remote determines ball direction.

3.2 Gameplay Operations

- User is prompted with a start screen where they can press a button on the remote to play the game
- We designed a FSM 4-state loop around bowling.
 - STATE_AIMING: Player moves left/right, and pressing the release button arms the throw based on the buttons on the remote.
 - STATE_ARMED: Release button is being held. The accelerometer is continuously sampled. It tracks peak swing force (power) and x-axis tilt (angle).
 - STATE_THROWING: Starts when the user lets go of the release button. Ball physics updates each tick (forward speed from power, lateral drift from angle) until it reaches the pins.
 - STATE_RESULT: Short pause after impact to show strike/spare feedback, then frame/throw bookkeeping runs (including 10th-frame bonus rules), and FSM returns to STATE_AIMING

4.0 Attribution Table

Partner	Fraction of Work	Description of Work
Ansh Shah	50%	• Created a bit-banged I2C protocol for the accelerometer • Created and wired the Wii remote circuit • Developed an Exponential Moving Average (EMA) filter for input noise reduction and smoothing • Swing power logic using 3D magnitude computation of accelerometer values & g-value conversion • Ball tilt angle logic using accelerometer values • Player left and right movement logic using pushbuttons on remote • Refined some VGA graphic aesthetics

		Helped fine tune the scoring/frame progression & throw animation
Bilaal Khan	50%	- Created and designed the FSM, including state transitions and result handling. - Contributed to core bowling game logic, including throw flow, pin interactions, and scoring/frame progression behavior. - Helped debug and tune accelerometer input processing. - Built fan crowd animation - Implemented animated lighting effects - Implemented crowd speech-bubble callouts with camera like flashes with timed/randomized behavior for spectator interaction. - Imported and integrated continuous audio playback. - Implemented the full VGA graphics